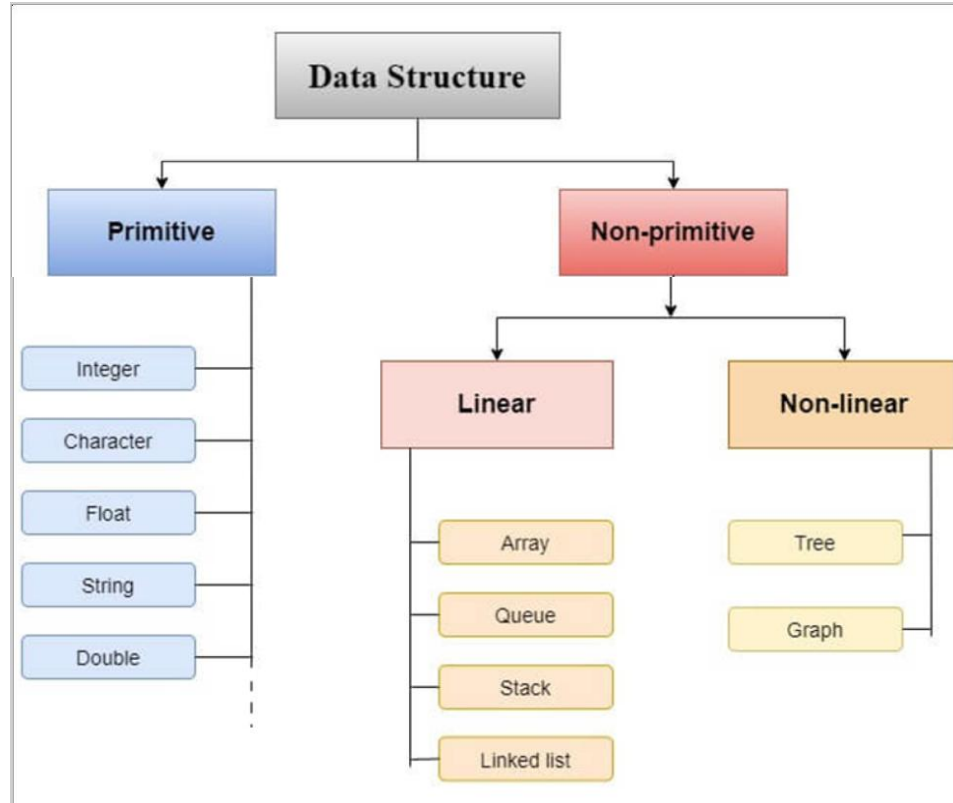


Lineaire datastructuren deel 1

Bachelor IT



Hoe zou je deze applicatie bouwen ?

```
Geef een aantal getallen in. Druk [enter] om te eindigen
```

```
10  
20  
300  
3  
56  
2
```

```
Hier zijn de getallen terug in dezelfde volgorde
```

```
10  
20  
300  
3  
56  
2
```

- Opdracht:

- Getallen bijhouden
- Het aantal is op voorhand niet gekend
- Volgorde moet ook onthouden worden.

=> **Queue** (= wachtrij) !





EOH

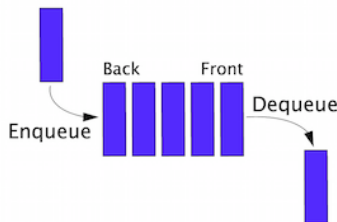
Queue ?

- Lineaire structuur: elementen staan op één rij.
- Volgorde blijft bewaard.
- Eerste **in** de queue = eerste **uit** de queue
⇒ **FIFO** principe
- Onmogelijk om gegevens in het midden van de rij eruit te halen. Eerst moeten alle voorgangers verwijderd worden.



Queue: bewerkingen

- 2 bewerkingen mogelijk:
 - Elementen toevoegen aan de queue: **Enqueue**
 - Elementen verwijderen uit de queue: **Dequeue**



- De rij kan dus groeien en terug kleiner worden, enz..
- Belangrijk: bij een **Dequeue** wordt het element dus uit de queue verwijderd !

Queue: eigenschappen

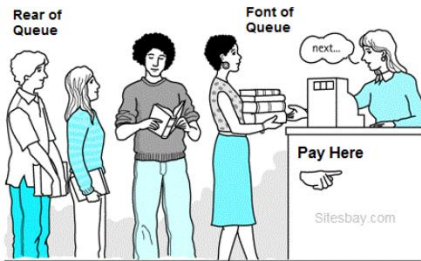
- Aantal elementen
- Maximum aantal elementen
⇒ Capaciteit van de queue

Volzet !



Bij sommige queues kan de capaciteit vanzelf groter worden

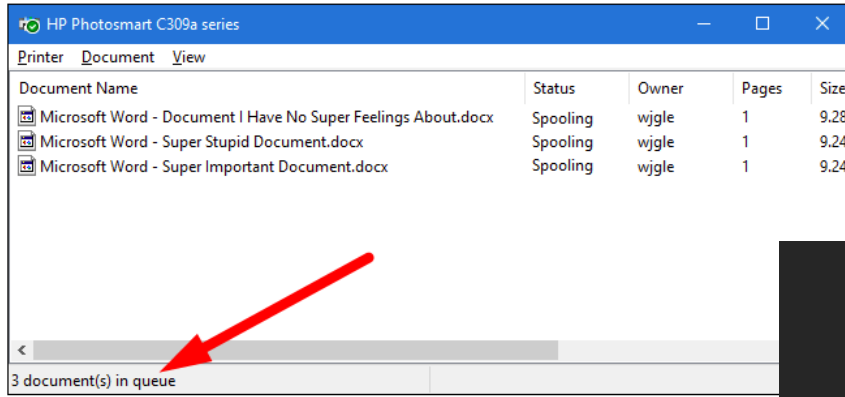
We kennen de queue reeds...



Real Life Example of Queue : Library Counter



IT Toepassingen van een queue

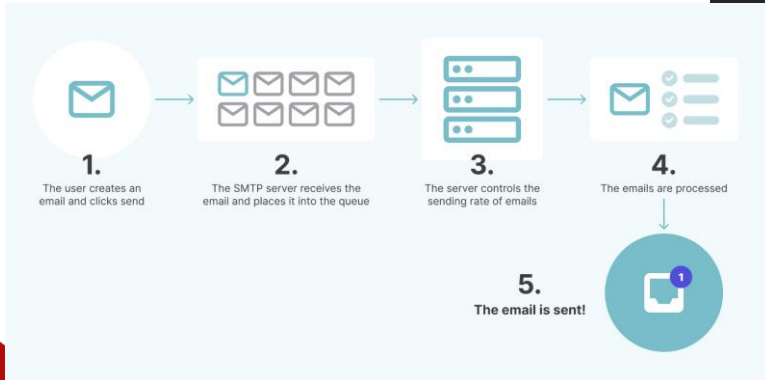


Preparing

REC-8235
REC-8236
REC-8238

Collect

REC-8234
REC-8237



ADELE LIVE 2016

Tuesday, March 08 2016
Manchester Arena
Manchester, United Kingdom

You are in a queue to purchase tickets due to the high demand.

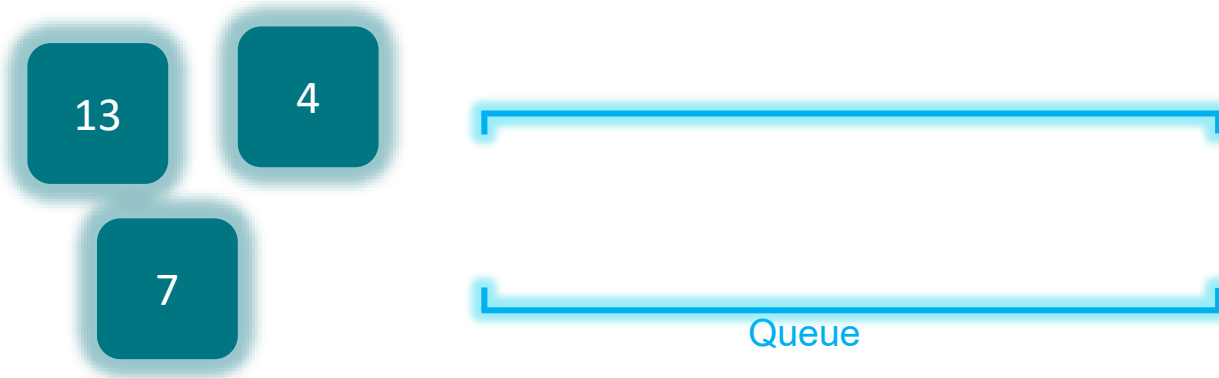
Please do not reload this page or you may lose your place in the queue.

Updating ticket availability for this event in 10 seconds.

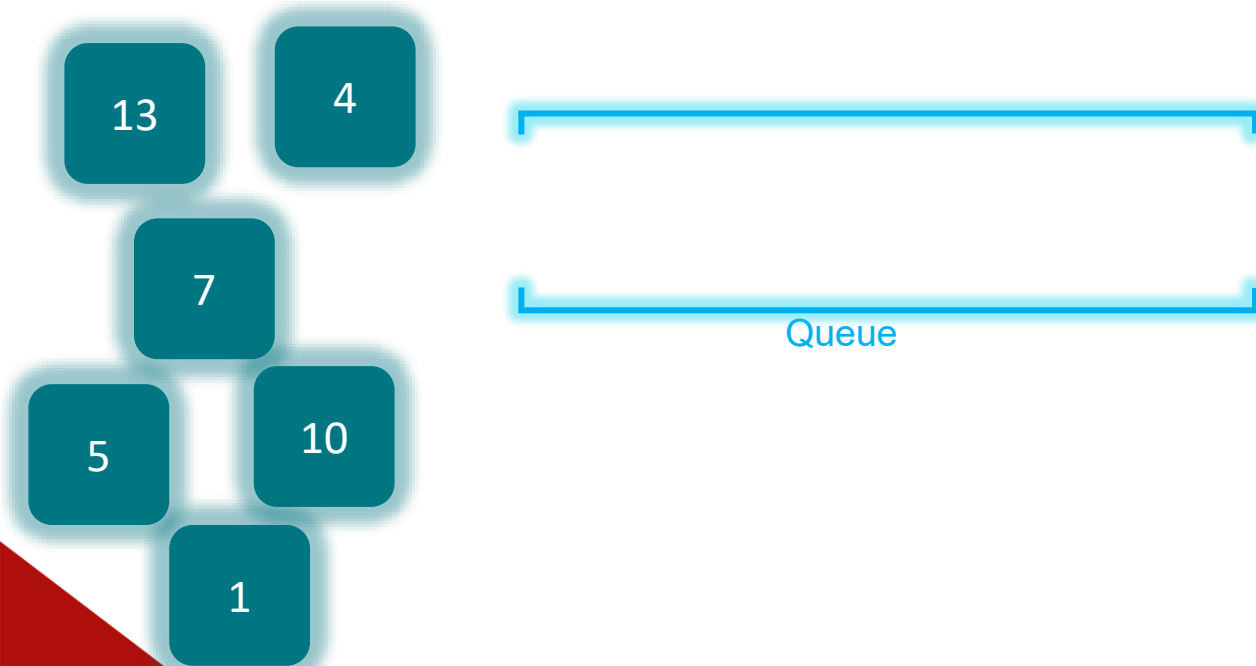
Tickets are currently: **still available**

Last updated 09:16:40

Werking van de Queue



Werking van de Queue



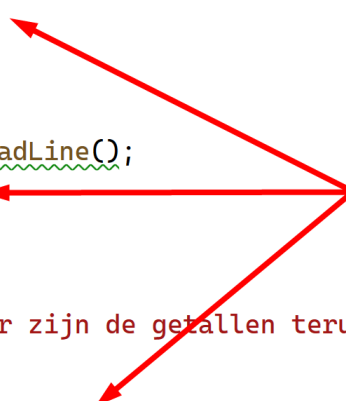
Queue gebruiken

```
private static void TestQueue()
{
    Console.WriteLine("Geef een aantal getallen in. Druk [enter] om te eindigen");

    string input = "";
    Queue q = new Queue();

    do
    {
        input = Console.ReadLine();
        q.Enqueue(input);
    }
    while (input != "");

    Console.WriteLine("Hier zijn de getallen terug in dezelfde volgorde");
    while(q.Count != 0)
    {
        Console.WriteLine(q.Dequeue());
    }
}
```



Queue zelf bouwen... ?

- OOP -> klasse -> instantie
- Methodes: **Enqueue()** & **Dequeue()**
- Properties: **Count**, **Capacity**, **IsFull**, **IsEmpty**,...
- Maar dan....
- In welke datastructuur gaan we de gegevens bewaren ?
- => **Array !**
- => Linked list (zie later)

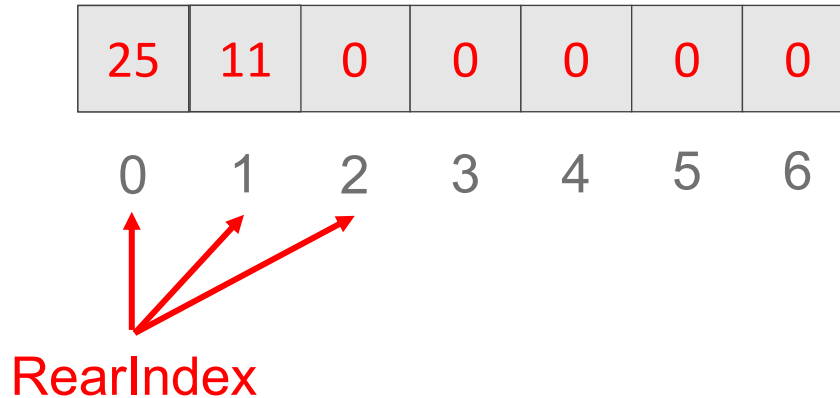
Queue zelf bouwen met een array

- Enqueue (25)
- Enqueue (11)
- Enqueue (33)
- Dequeue() 25
- Dequeue() 11
- Enqueue (70)
- Enqueue (0) ?

0	0	0	0	0	0	0
25	0	0	0	0	0	0
25	11	0	0	0	0	0
25	11	33	0	0	0	0
11	33	0	0	0	0	0
33	0	0	0	0	0	0
33	70	0	0	0	0	0

Queue zelf bouwen....

- We hebben nodig:
 - 1 array (grootte => capaciteit van de queue)
 - 1 hulp variabele: **RearIndex** (achterzijde van de queue)



Queue zelf bouwen met een array

- Enqueue (25)
- Enqueue (11)
- Enqueue (33)
- Dequeue() 25
- Dequeue() 11
- Enqueue (70)
- Enqueue (0)

0	1	2	3	4	5	6	RearIndex
0	0	0	0	0	0	0	-1
25	0	0	0	0	0	0	0
25	11	0	0	0	0	0	1
25	11	33	0	0	0	0	2
11	33	33	0	0	0	0	1
33	33	33	0	0	0	0	0
33	70	33	0	0	0	0	1
33	70	0	0	0	0	0	2



EOH

Queue zelf bouwen

- Lineair Queue (enkel RearIndex)
- Nadeel ?
 - Bij Dequeue(): alle elementen 1 plaats opschuiven
 - Kost tijd bij heel grote queues (vergelijk met het “jojo” effect bij een file op de autostrade)
- Oplossing ?
 - **Circular** Queue (FrontIndex & RearIndex)

Circular queue

FrontIndex RearIndex

	0	1	2	3	4	5	6		
	0	0	0	0	0	0	0		
Enqueue(25)	25	0	0	0	0	0	0	0	0
Enqueue(11)	25	11	0	0	0	0	0	0	1
Enqueue(33)	25	11	33	0	0	0	0	0	2
Dequeue()	25	11	33	0	0	0	0	1	2
Dequeue()	25	11	33	0	0	0	0	2	2

Beperking van een **queue** op basis van een **array** ?

- **Grootte van de array** moet bij de start worden gekozen
 - Array **te groot** in verhouding tot het gebruik => verspilling van geheugen
 - Array **te klein** => queue full
- **Oplossing ?**
- Queue volzet ?
 - => Nieuwe array aanmaken en alle data over kopiëren

Hoe zou je deze applicatie bouwen ?

```
Geef een aantal getallen in. Druk [enter] om te eindigen
```

```
10  
50  
80  
95  
120  
33
```

```
Hier zijn de getallen terug in omgekeerde volgorde
```

```
33  
120  
95  
80  
50  
10
```

- Opdracht:

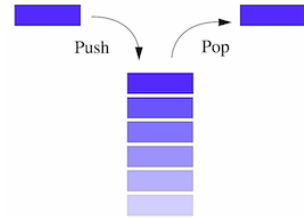
- Getallen bijhouden
- Het aantal is op voorhand niet gekend
- Volgorde moet **omgekeerd** worden.

=> **Stack** (= stapel) !



Stack ?

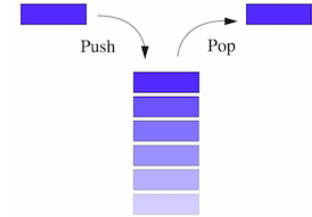
- Lineaire structuur: elementen staan op één rij.
- Volgorde blijft bewaard.
- **Laatste** in de stack = **eerste** uit de stack
⇒ **LIFO** principe
- Onmogelijk om gegevens in het midden van de rij eruit te halen. Eerst moeten alle voorgangers verwijderd worden.



Stack: bewerkingen



- 2 bewerkingen mogelijk:
 - Elementen toevoegen aan de stack: **Push**
 - Elementen verwijderen uit de stack: **Pop**



- De rij kan dus groeien en terug kleiner worden, enz..
- Belangrijk: bij een **Pop** wordt het element dus uit de stack verwijderd !
- 3^e bewerking: **Peek** (kijken welk element bovenaan de stapel ligt zonder het te verwijderen)

Stack: eigenschappen

- Aantal elementen
- Maximum aantal elementen
⇒ Capaciteit van de stack



Bij sommige stacks kan de capaciteit vanzelf groter worden

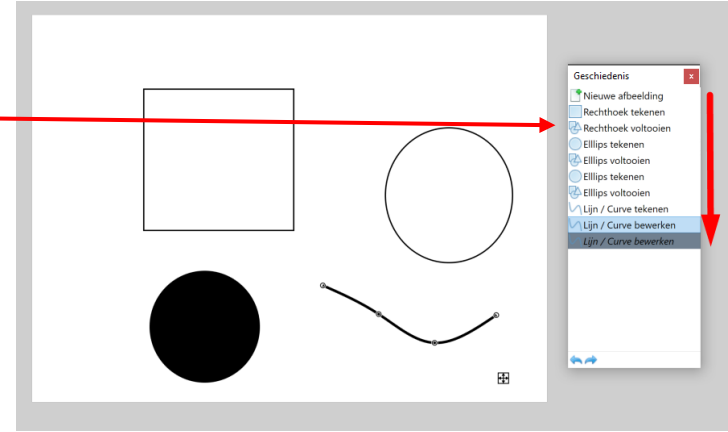
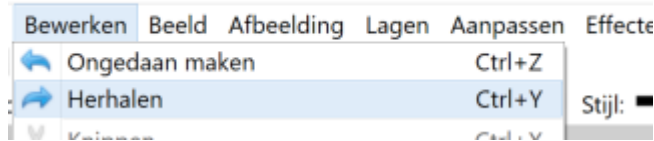
We kennen de stack reeds...



IT Toepassingen van een stack

Undo functionaliteit (Paint, Word, Excel ..)

loos - paint.net 4.0.19



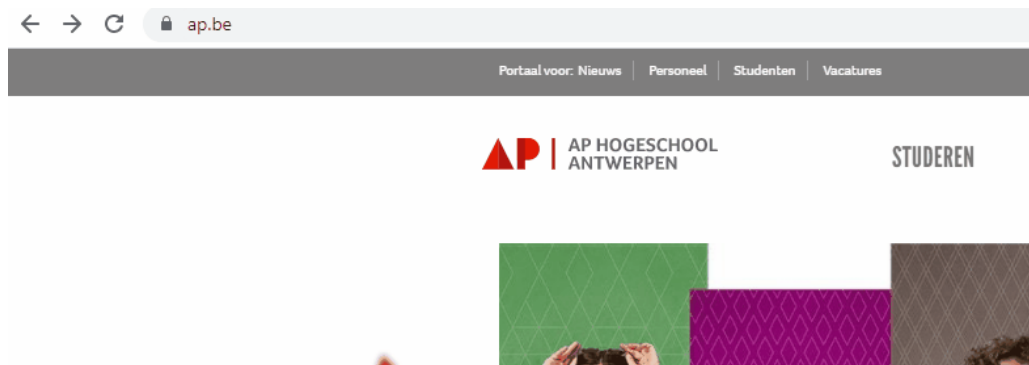
Backward & Forward in de browser



AP | AP HOGESCHOOL
ANTWERPEN



“Back”-stack in de browser



Huidige url: **ap.be**

Url Stack



Call-Stack bij het uitvoeren van een c# applicatie

```
0 references
public MainWindow()
{
    InitializeComponent();

    string zin = VerwijderSpaties("Dit Is EEn ZIN MET GROTE en kleine letters en spaties");
}

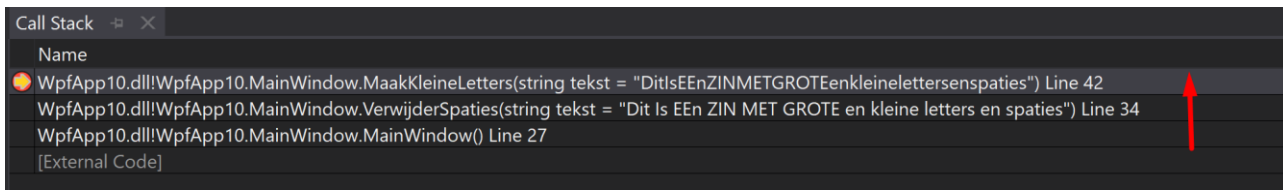
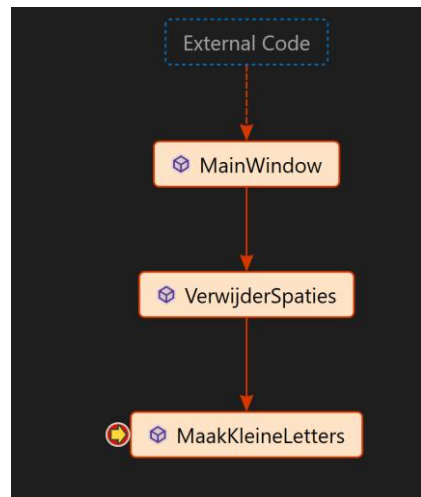
1 reference
string VerwijderSpaties(string tekst)
{
    string TekstZonderSpaties = tekst.Replace(" ", "");

    string TekstZonderSpatiesEnKleineLetters = MaakKleineLetters(TekstZonderSpaties);

    return TekstZonderSpatiesEnKleineLetters;
}

1 reference
string MaakKleineLetters (string tekst)
{
    string KleineLetters = tekst.ToLower();

    return KleineLetters;
}
```



Stack: Controleren / Berekenen van expressies

- bv: $\{a^2 - [(b + c)^2 - (d + e)^2] * [\sin(x - y)]\} - \cos(x + y)$
- De haakjes moeten steeds in balans zijn !

{[(())][()]}()

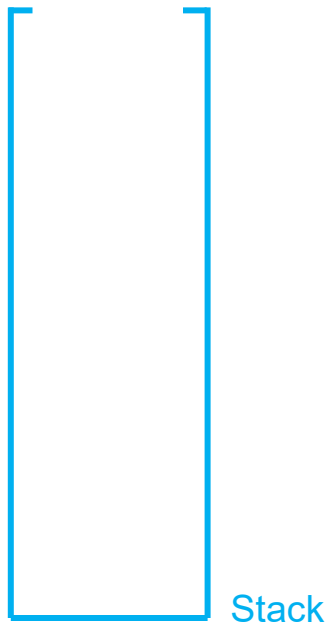
- Kan gecontroleerd worden mbv. een stack
 - Haakje open : push op stack
 - Haakje sluit : pop van stack en vergelijk het type
 - { [(

Werking van de Stack

13

7

4

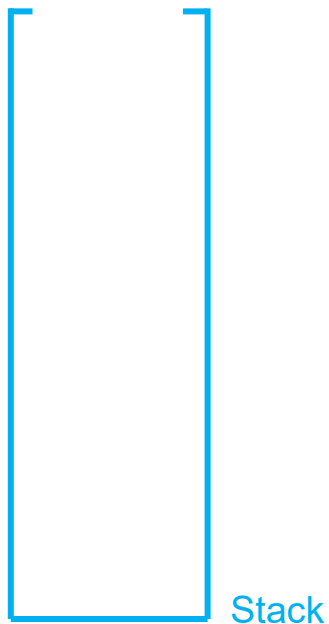


Werking van de Stack

13

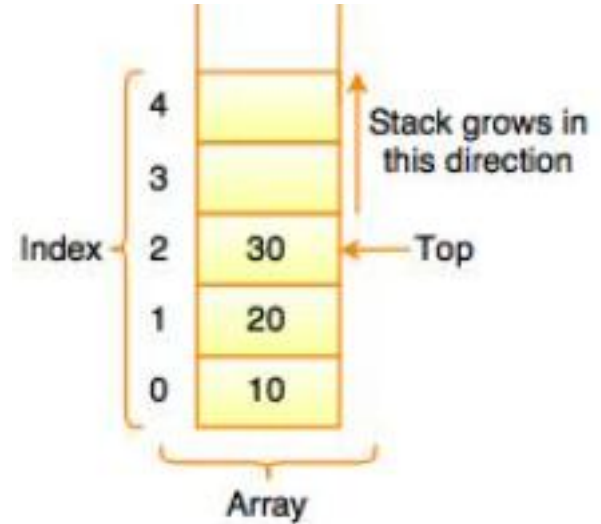
7

4



Hoe stack implementeren adhv. array ?

- **Array** (bepaalt ook ineens de capaciteit van de stack)
- **“Top” variabele** die de index aangeeft van het bovenste element.
- Deze variabele zal mee verhogen / verlagen bij een push / pop
- Check : Array Leeg / Volzet !



Beperking van een **stack** op basis van **Array** ?

- **Grootte van de array** moet bij de start worden gekozen
 - Array **te groot** in verhouding tot het gebruik => verspilling van geheugen
 - Array **te klein** => stackoverflow
- **Oplossing ?**
- Stack volzet ? -> Nieuwe array aanmaken en alle data over kopiëren
 - => oefeningen

Stack of Queue ?



